МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ПРОГРАМНИХ СИСТЕМ ТА ПРИКЛАДНОЇ МАТЕМАТИКИ

Кафедра: СПСКС

ЗВІТ

про виконання лабораторної роботи №1

з дисципліни «Бази даних та засоби управління»

тема: «Проектування бази даних та ознайомлення з базовими операціями СУБД PostgreSQL»

Виконав: ст. гр. КВ-41

Ткач Дімітрій

Перевірив: доц. каф. СПСКС, к.т.н.

Петрашенко А.В.

Київ 2026

Предметна галузь: Онлайн-магазин електроніки.

Опис предметної галузі.

Онлайн-магазин електроніки - це галузь, архітектура бази даних якої першочергово потребує:

1) зберігання інформації про Товари та пов’язаних з ними даних(наприклад: наявна кількість, вартість, фізичні характеристики, найменування тощо);
2) інформація про Користувачів(наприклад: ПІБ, дата створення профілю, контакти, адреса доставки тощо);
3) інформація про Замовлення (наприклад: інформація про Замовника/Користувача, інформація про Товар, кількість, дата замовлення, адреса доставки, тощо);
4) також архітектура БД повинна врахувати що користувач не завжди шукатиме конкретний товар, навпаки, у більшості випадків користувач знаєш лише Категорію якою можна класифікувати його потребу.

Примітка: існує багато магазинів які можуть працювати лише з зареєстрованими в їх базі користувачами, також є і такі що можуть потребувати їх даних лише для оформлення доставки замовленого товару, а є і ті, які містять інформацію про товари та адресу фізичного магазину де їх можна забрати, а інформацію про користувача зберігають лише на час виконання замовлення яке часто включає не так багато інформації про користувача, як про номер його замовлення.

Для визначеності, у даній роботі, було прийнято рішення, що обрана предметна галузь належить саме до тих хто якимось чином взаємодіє з клієнтами лише, якщо вони зареєстровані у їх базі даних.

Отже підсумовуючи опис предметної галузі, онлайн-магазин електроніки працює так: користувач реєструється у БД, шукає те що його цікавить вводячи конкретну назву товару чи знаючи лише його категорію і шукає серед списку доступних, після вибору, він оформляє замовлення яке включає: один або декілька обраних товарів, після чого архітектура фіксує цей зв'язок, всю необхідну інформацію про замовлення та передає його в обробку.

Посилання на репозиторій GitHub.
https://github.com/DimitriiTkachreal/db-course-KPI

Контакт в Телеграм.

@tkachd7

Виконання завдання. Пункт № 1.

Перелік сутностей з описом їх призначення.

1) Items - зберігає інформацію про доступні у магазині товари. Це незалежна сутність, що містить актуальні дані для формування замовлень.
   Структура цієї сутності попередньо планується так:

   ItemName - назва людською мовою, за якою користувач може здійснювати пошук конкретного товару/категорії товарів.

   Quantity - кількість наявних для продажу одиниць товару. Допоможе сформувати логіку, яка не дозволить здійснити замовлення товару, якого немає у продавця тощо.

Cost - вартість одиниці конкретно цього товару. Допоможе сформувати логіку транзакцій оплати залежно від кількості замовленого товару.

Тобто підсумовуючи опис сутності Items, це структура що містить актуальну дані про товари що продаються, та які необхідні для формування замовлень та логіки їх обробки.

2) Users - зберігає актуальні дані користувача необхідні для адміністрування у онлайн-магазині електроніки.

Структура цієї сутності попередньо планується так:

Email - електронна пошта, яка слугує унікальним логіном та засобом зв'язку.

FullName - ПІБ користувача одним рядком.

Password - пароль для входу у профіль сайту магазину.

CreationDate - дата формування профілю.

Підсумовуючи опис сутності Користувач, це структура, що зберігає базову необхідну інформацію про користувача, яку можна використати для формування замовлення.

3) Сутність Orders - фіксує факт здійснення покупки конкретним користувачем.

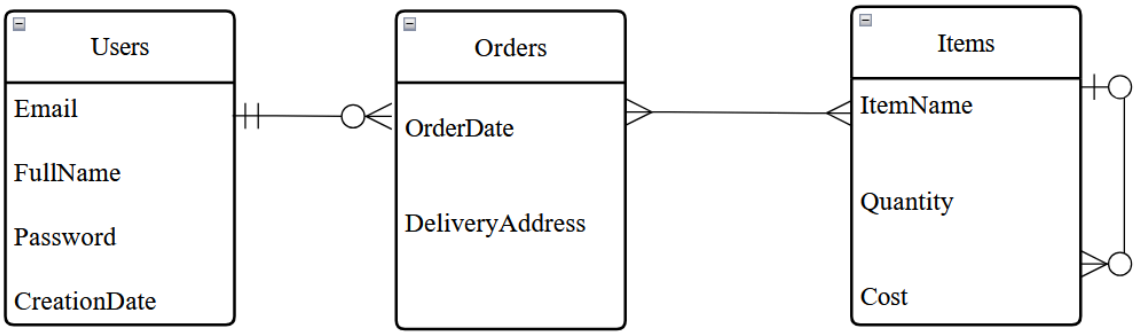
Структура сутності попередньо планується такою:

OrderDate - дата та час оформлення замовлення.

DeliveryAddress - сформована адреса доставки для цього конкретного замовлення одним рядком.

Підсумовуючи опис сутності Orders, це структура, яка об'єднує користувача з фактом покупки, фіксуючи час і місце доставки, але при цьому не перевантажується зайвими деталями про самі товари.

ER-модель бази даних.



Мал.1. ER-модель баз даних онлайн магазину електроніки.

Модель розроблено з дотриманням правил та стилю Crow's Foot Notation.

Виконання завдання. Пункт № 2.

Опис процесу перетворення ER-моделі на схему бази даних.

Перетворення сутності Items.

Згідно опису цієї сутності це структура даних що містить дані про товари, а саме: ім’я, кількість та вартість одиниці. Вміст не зберігає і жодним чином не залежить від інших таблиць , тобто вона не містить у своєму шаблонів зовнішніх ключів на інші таблиці. Проте як видно з моделі, у цій структурі присутній унарний зв’язок із самою собою, оскільки у магазині передбачається наявність різних категорії.

Отож підсумовуючи наявні дані про цю сутність, пропонується така структура таблиці:

PK: INTEGER ArticleID - ціле число що характеризує конкретно цей товар, воно є унікальним для нього, та дозволяє швидко знайти його, та формувати зв’язок з іншою сутністю.

FK : INTEGER ParentID - зовнішній ключ що вказує на батьківську категорію, у цій самій таблиці.

NOT NULL : TEXT ItemName - текстова назва товару/категорії.

INTEGER Quantity - цілочисельне значення кількості товарів на продаж.

INTEGER Cost - одинична вартість товару.

Перетворення сутності Users.

Згідно опису цієї сутності випливає, що це структура даних що містить дані про користувачів, а саме: ПІБ, електронна пошта, пароль та дату створення профілю. Вміст не зберігає і жодним чином не залежить від інших таблиць, тобто вона не містить у своєму шаблоні зовнішніх ключів на інші таблиці.

Отож підсумовуючи наявні дані про цю сутність, пропонується така структура таблиці:

PK : INTEGER UserID - унікальний номер створеного користувачем профілю у БД магазину.

UNIQUE NOT NULL : TEXT Email - електронна пошта, яка слугує унікальним логіном та засобом зв'язку. Оскільки не можна допустити щоб за однією електронною поштою було два профілі, накладено правило унікальності запису.

NOT NULL : TEXT FullName - ПІБ користувача.

NOT NULL : TEXT Password - пароль для входу у профіль.

NOT NULL : DATE CreationDate - дата формування профілю.

Перетворення сутності Orders.

Згідно опису цієї сутності це структура даних що містить дані про замовлення, а саме: дату замовлення та адресу доставки.

Проте очевидно що для адміністрування замовлення, цих даних недостатньо. З ER-моделі випливає що ця сутність також має поєднати в собі зв’язок між тим хто оформляє замовлення, тобто користувач, та тим що він власне хоче купити в цьому замовленні, тобто конкретний товар та його кількість.

Отож підсумовуючи наявні дані про цю сутність, пропонується така структура таблиці:

PK : INTEGER OrderID - первинний ключ рядка конкретного замовлення.

FK NOT NULL : INTEGER UserID - зовнішній ключ, який вказує на користувача, що оформив замовлення. Правило не-нуль накладено бо замовлення без користувача не може бути зареєстроване.

NOT NULL : DATE OrderDate - дата та час оформлення замовлення.

NOT NULL : TEXT DeliveryAddress - сформована адреса доставки для цього конкретного замовлення одним рядком.

Примітка: як видно з моделі сутність Orders утворює з сутністю Items зв’язок M:N, що логічно, адже одне замовлення може містити в собі багато товарів, так і один товар може належати до багатьох замовлень.

Отже для реалізації цього зв’язку пропонується створити сутність-посередник, OrderItem.

Структура таблиці OrderItem:

FK NOT NULL : INTEGER OrderID - зовнішній ключ, посилання на конкретне замовлення.

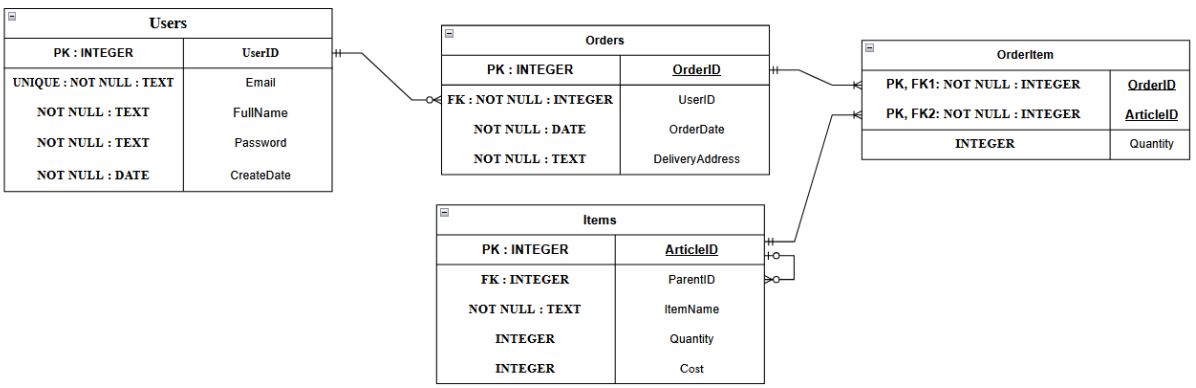
FK NOT NULL : INTEGER ArticleID - зовнішній ключ, посилання на конкретний товар у цьому замовленні.

Примітка: первинний ключ у цій таблиці формується з комбінації зовнішніх ключів OrderId та ArticleId, цим самим гарантуючи унікальність кожного запису та запобігаючи дублюванню одного й того самого товару в межах одного замовлення.

NOT NULL : INTEGER Quantity - кількість одиниць конкретного товару саме в цьому замовленні.

Підсумовуючи опис сутності OrderItem, вона дозволяє додавати безліч товарів в один чек, і містить атрибут зв'язку Quantity, який показує, скільки одиниць конкретного товару було куплено.

Схема таблиць бази даних.



Мал.2. Схема таблиць баз даних онлайн магазину електроніки.

Виконання завдання. Пункт № 3.

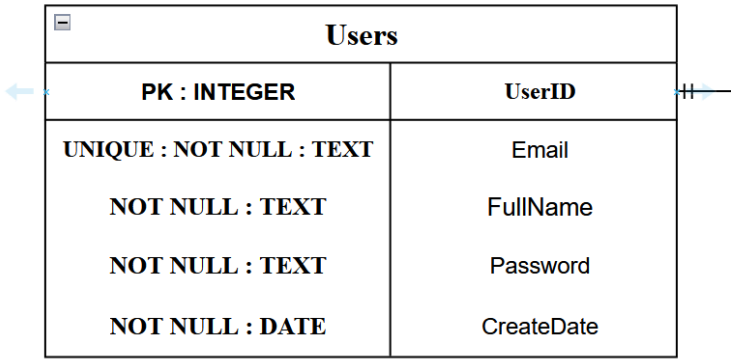
Аналіз реляційної схеми на відповідність НФ.

Аналіз таблиць.1НФ.

Згідно визначення першої нормальної форми, схема бази даних вважається їй відповідною, якщо дані в таблицях є атомарними, а рядки унікальними.

Примітка: як видно з схеми таблиць баз даних, у кожній таблиці присутній свій первинний ключ, який забезпечує унікальність кожного окремого запису в таблиці. Тому надалі в аналізі буде перевірятися лише атомарність кожної таблиці.

Аналіз таблиці Users.



Малюнок.3. Схема таблиці Users.

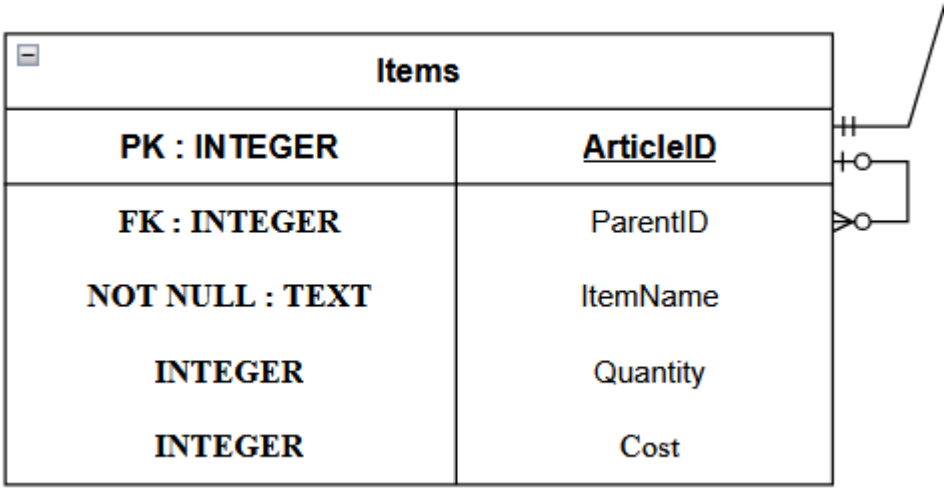
Атрибути Email, Password, CreateDate - зберігає текстовий запис електронної пошти користувача паролю та дати створення профілю. Оскільки логічно неможливо розбити ці поняття на менш загальні, випливає що атрибути є атомарними.

Як видно з мал.3. на атрибут Email накладено правило унікальності, це зроблено з метою уникнення реєстрації користувачів з однаковою поштою, оскільки такого бути не може, натомість атрибут FullName такого

правила немає оскільки досить часто трапляються люди з однаковим іменем, прізвищем, та інколи навіть по-батькові.

Атрибут FullName - він зберігає текстовий запис повного імені користувача, це просто один великий рядок. В межах пропонованої архітектури бази даних для обраної предметної галузі, було прийнято рішення що вся база даних буде оперувати саме повним ім’ям, завжди за будь-яких умов, оскільки в цій роботі були встановлені значні обмеження на кількість атрибутів та сутностей, то для відповідності сутності 1НФ, треба або зменшити кількість інформації про користувача, де її і без цього внесений базовий мінімум необхідний для більш-менш зручного адміністрування, або встановити що в межах цієї архітектури ПІБ це поняття атомарне та не роздільне, було обране другий спосіб, оскільки зазвичай при оформленні замовлення у будь-якому сервісі, треба вказувати своє ім’я. Якщо б обмежень не було найкращим рішенням було б винести дані користувача як окрему людину в окрему таблицю баз даних.

Аналіз таблиці Items.



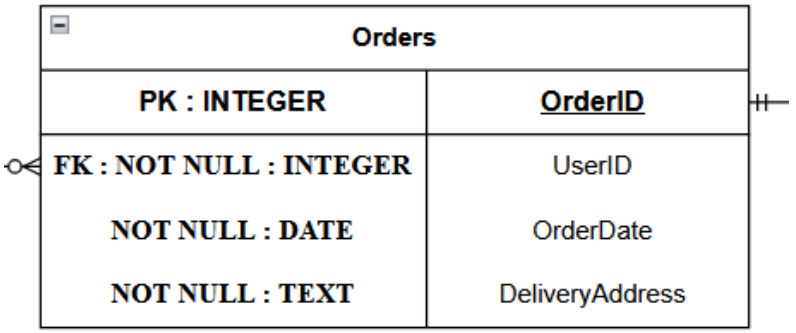
Малюнок.4. Схема таблиці Items.

Атрибут ItemName містить логічно атомарну назву конкретного товару або категорії, яку неможливо і недоцільно розбивати на менші складові в межах даної предметної галузі.

Як було обґрунтовано в описі предметної галузі, архітектура бази даних онлайн-магазину розробляється так, що таблиця Items одночасно зберігає як кінцеві товари, так і категорії для їх класифікації. Для реалізації цієї логіки в таблицю було введено атрибут ParentID який вказує на первинний ключ іншого рядка. На атрибути Quantity та Cost не накладались жодні правила, завдяки чому їх значення можуть бути NULL.

Таким чином, кожен окремий атрибут у будь-якому рядку таблиці Items містить строго одне скалярне значення.

Аналіз таблиці Orders.



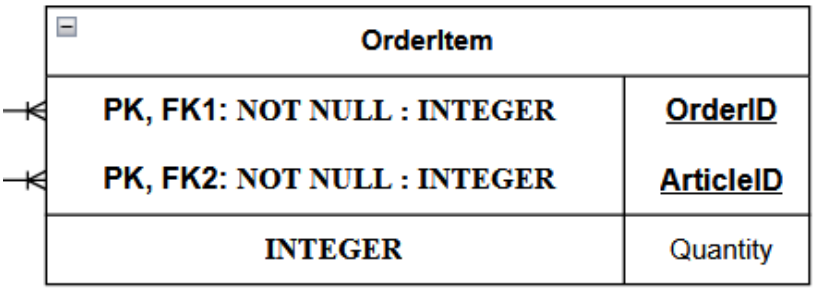
Малюнок.5. Схема таблиці Orders.

Атрибути UserID, OrderDate, DeliveryAddress - зберігають цілочисельний ідентифікатор користувача, дату та час оформлення покупки, а також текстовий запис сформованої адреси доставки. Оскільки ці дані за своєю природою описують прості скалярні величини, які неможливо розбити на менші значущі компоненти в межах поточної бізнес-логіки системи, вони є повністю атомарними.

Як видно з мал.5. на атрибут UserID накладено правило зовнішнього ключа з обмеженням не-нуль. Це зроблено тому, що замовлення логічно не може існувати без покупця, який його створив, і архітектура бази даних повинна контролювати цей зв'язок щоб зберегти цілісність.

Атрибут DeliveryAddress зберігає всю адресу доставки одним суцільним великим рядком. Подібно до ситуації з повним іменем користувача, у цій архітектурі було прийнято рішення не дробити адресу на окремі стовпці міста, вулиці чи індексу. З урахуваннням лімітів на загальну кількість атрибутів у сутностях, об'єднання адреси в один рядок є оптимальним компромісом, який зберігає інформативність і не перевантажує структуру зайвими стовпцями.

Аналіз таблиці OrderItem.



Малюнок.6. Схема таблиці OrderItem.

Атрибути OrderID, ArticleID, Quantity - зберігають цілочисельний ідентифікатор замовлення, ідентифікатор конкретного товару та кількість одиниць цього товару в чеку. Усі три поля містять прості скалярні значення числового типу, які логічно неможливо розділити на дрібніші компоненти, що прямо вказує на їхню повну атомарність.

Аналіз таблиць.2НФ.

Згідно визначення 2НФ, можна зробити висновок що, сенс перевіряти таблицю на її відповідність цій нормальні формі, є лише для тих таблиць, у яких первинний ключ є складеним. Усі інші таблиці якщо вони задовольняють умови 1НФ, то автоматично задовольняють умови 2НФ.

Отже, оскільки всі таблиці в межах цієї архітектури баз даних відповідають умовам 1НФ, то залишилось перевірити відповідність таблиць з складеними атрибутами, тут така таблиця лише одна - OrderItem.

Аналіз таблиці OrderItem.2НФ.

Як видно з мал.6. таблиця має лише один неключовий атрибут, Quantity що характеризує кількість конкретного товару в конкретному замовленні. Якщо перевірити залежність поля Quantity від частини ключа, то випливають такі логічні висновки. Знаючи лише номер замовлення OrderID, неможливо визначити кількість, оскільки в одному чеку може бути багато різних товарів, і кожен з них має свій власний обсяг купівлі. Так само, знаючи лише ідентифікатор товару ArticleID, неможливо дізнатися його кількість, оскільки цей товар продається у різних обсягах у багатьох різних замовленнях. Значення кількості Quantity стає відомим і визначеним лише тоді, коли ми беремо повну комбінацію: конкретний номер замовлення та конкретний товар у ньому.

Отже, неключовий атрибут Quantity повно функціонально залежить від усього складеного первинного ключа.

Аналіз таблиць.3НФ.

Згідно визначення 3НФ, усі таблиці повинні відповідати 2НФ, а всі неключові атрибути не повинні мати залежностей між собою, а визначатися лише своїм первинним ключем.

Аналіз таблиці Users.

У цій таблиці первинним ключем є UserID. Неключовими атрибутами є Email, FullName, Password та CreationDate. Якщо проаналізувати логіку цих атрибутів, то можна зробити висновок, що електронна пошта, повне ім'я, пароль чи дата реєстрації ніяк не пов'язані між собою напряму. Зміна пароля користувача не тягне за собою автоматичну зміну його імені, а дата створення профілю ніяк не визначає його пошту, і навпаки. Кожен із цих атрибутів описує користувача незалежно від інших стовпців, і залежать тільки від UserID.

Аналіз таблиці Orders.

Первинним ключем цієї таблиці є OrderID, а не ключовими полями є зовнішній ключ UserID, а також атрибути OrderDate та DeliveryAddress.

Як було обгрунтовано раніше, атрибут адреси є суцільним одним рядком, то ж транзитивної залежності на кшталт Місто -> Країна в межах прийнятої для цієї архітектури бізнес логіки, немає. Атрибут дата замовлення ніяким логічним чином не пов’язаний з атрибутом місця доставки.

Отже, оскільки транзитивні залежності відсутні,то таблиця Orders відповідає 3НФ.

Аналіз таблиці Items.

У таблиці Items первинним ключем є ArticleID. Неключові атрибути це ItemName, Quantity, Cost та ParentID. Назва товару, його ціна та наявність на складі, є незалежними характеристиками об'єкта. Ціна товару ніяк не розраховується автоматично на основі його назви, а кількість на складі не визначає вартість одиниці.

Унарного зв'язку моделює ієрархічне дерево всередині таблиці, але це посилання на інший рядок, воно ніяк не керує значеннями сусідніх стовпців у поточному рядку. Назва, ціна та кількість позиції з ідентифікатором ArticleID описують саме цю позицію, без жодних транзитивних зв'язків між не ключовими полями.

Аналіз таблиці OrderItem.

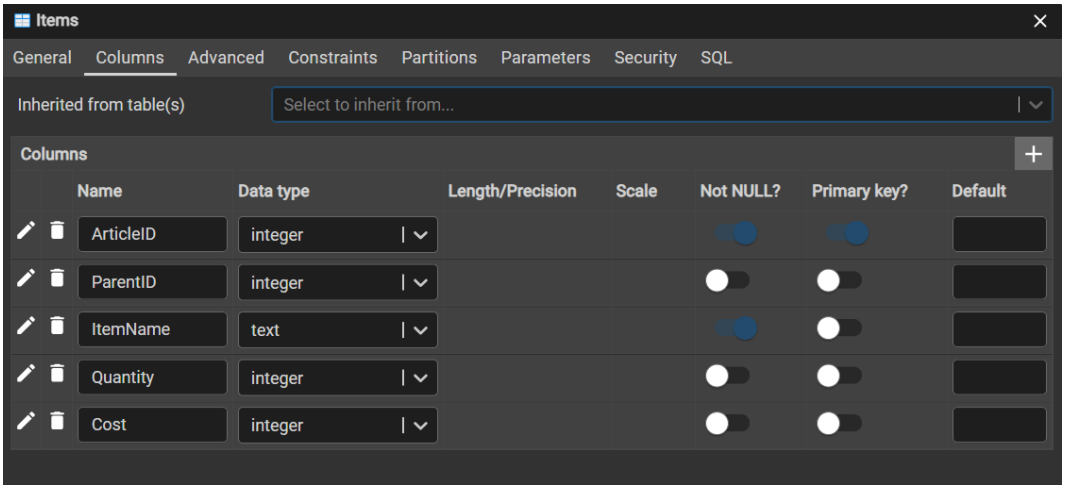
Ця таблиця має складений первинний ключ OrderID та ArticleID, а єдиним не ключовим атрибутом є Quantity, у нього фізично немає жодного іншого не ключового поля, від якого він міг би залежати чи якось взаємодіяти в межах одного рядка. Атрибут кількості залежить строго від усього складеного ключа, а транзитивні залежності відсутні в принципі.

Отже підсумовуючи аналіз схеми таблиць бази даниї онлайн магазину електроніки, випливає що пропонована архітектура цілком відповідає 1, 2 та 3 нормальним формам, а отже можна перейти до реалізації цієї схеми засобами PostgreSQL.

Виконання завдання. Пункт № 4.

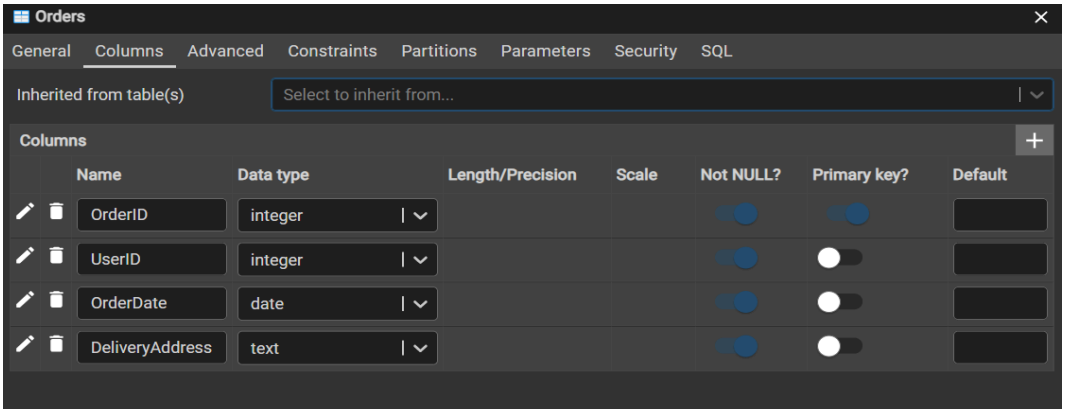
Властивості таблиць у pgAdmin4.

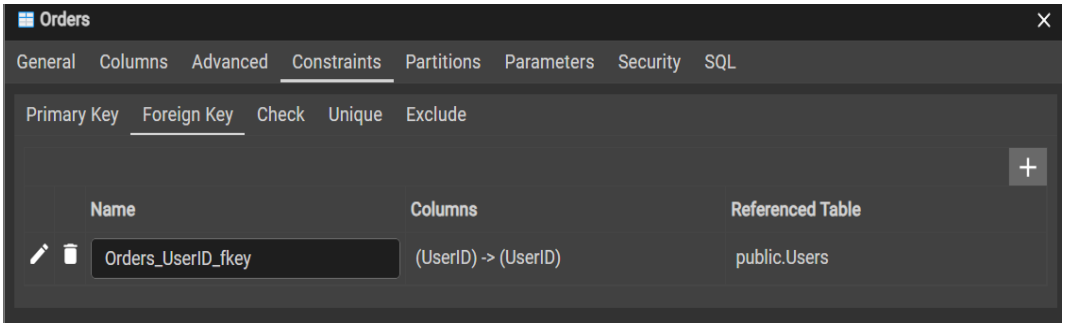
Таблиця Items.



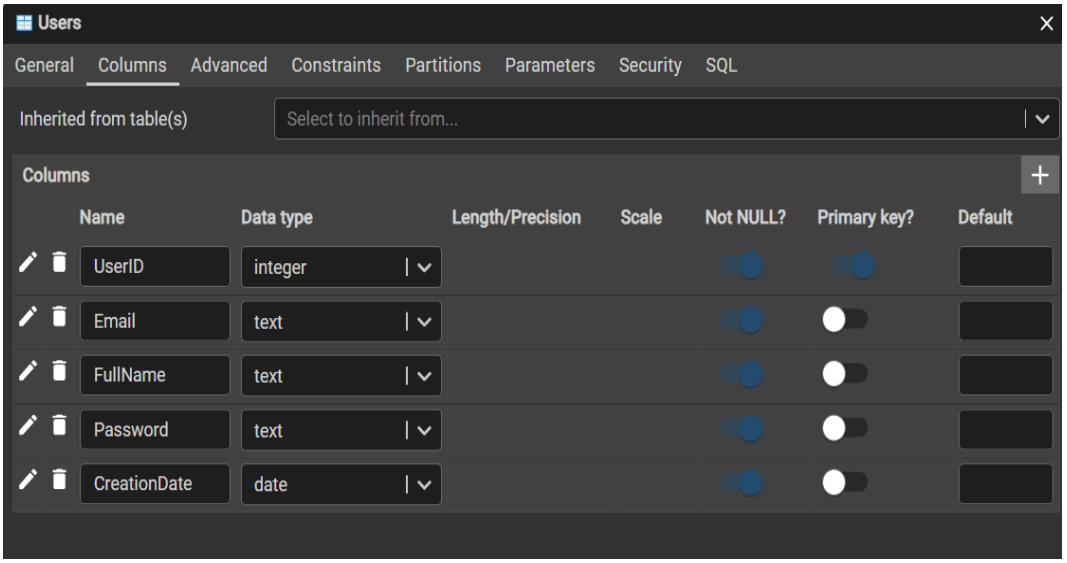


Таблиця Orders.

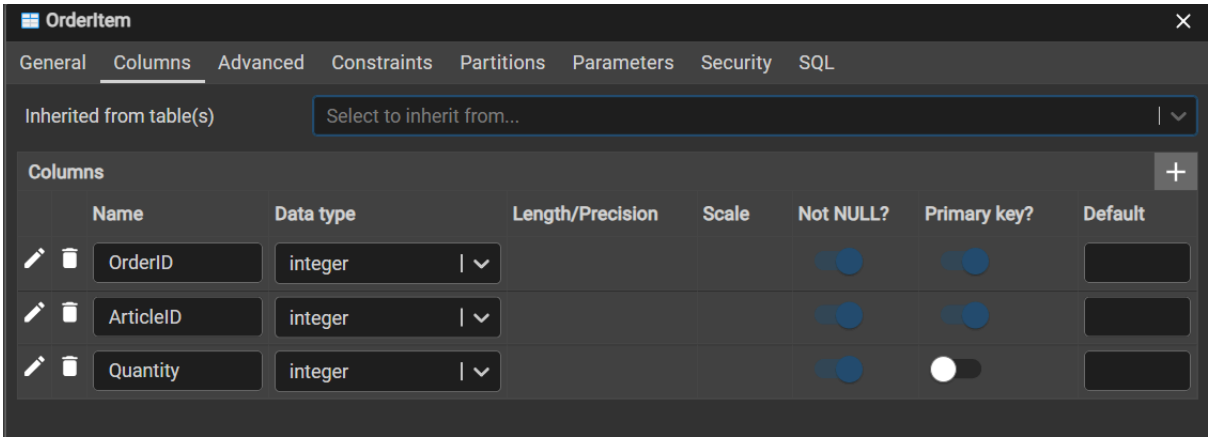


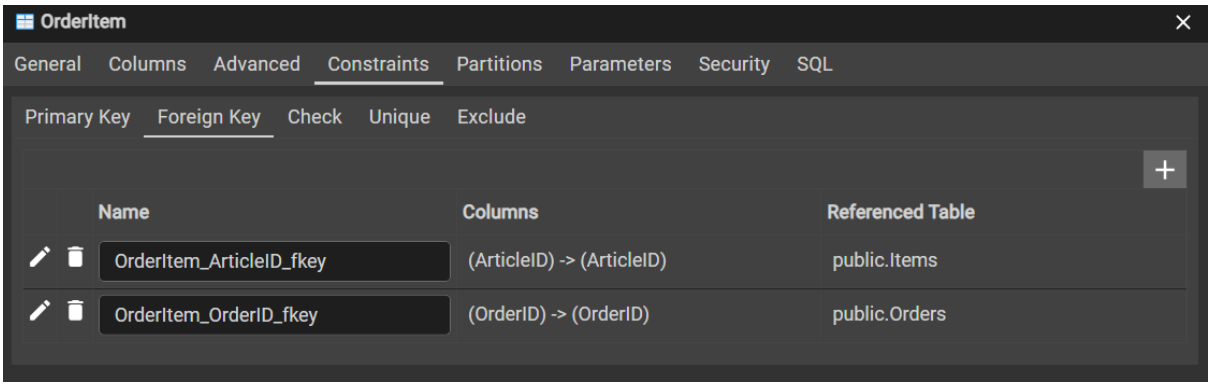


Таблиця Users.



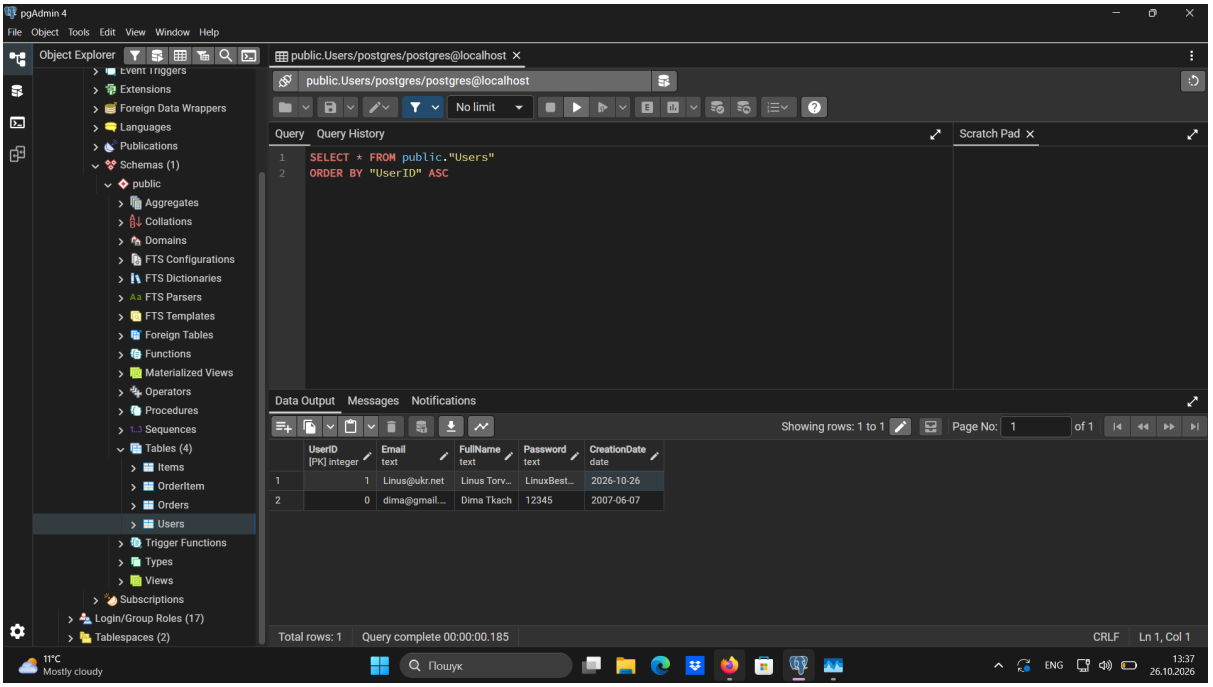
Таблиця OrderItem.



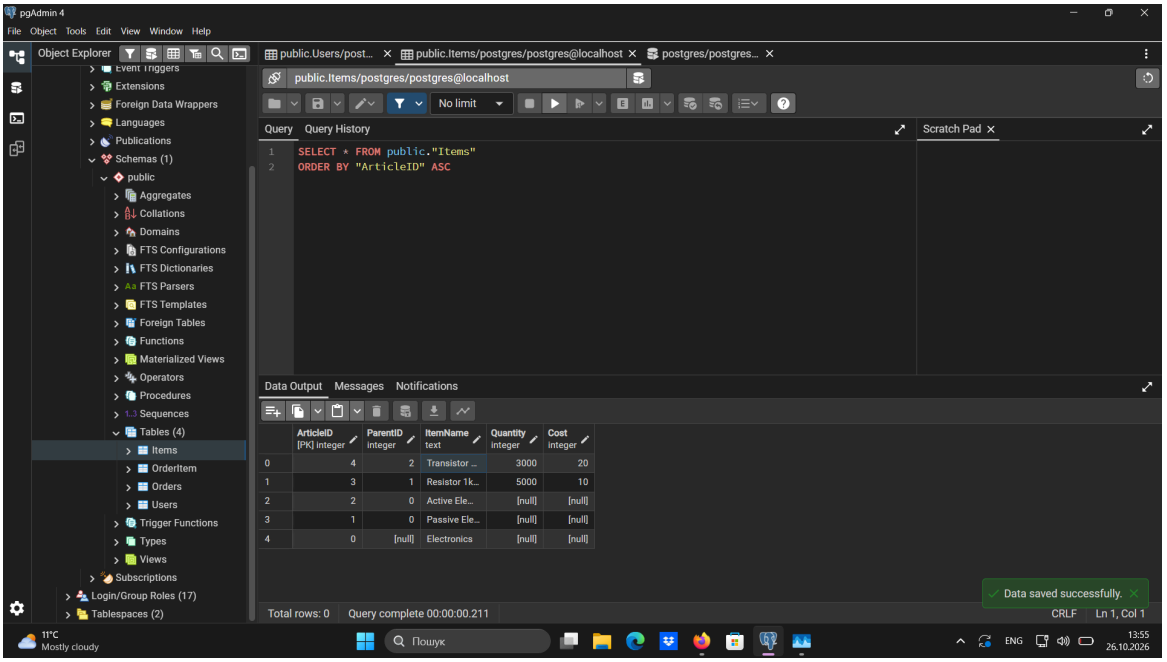


Вміст таблиць.

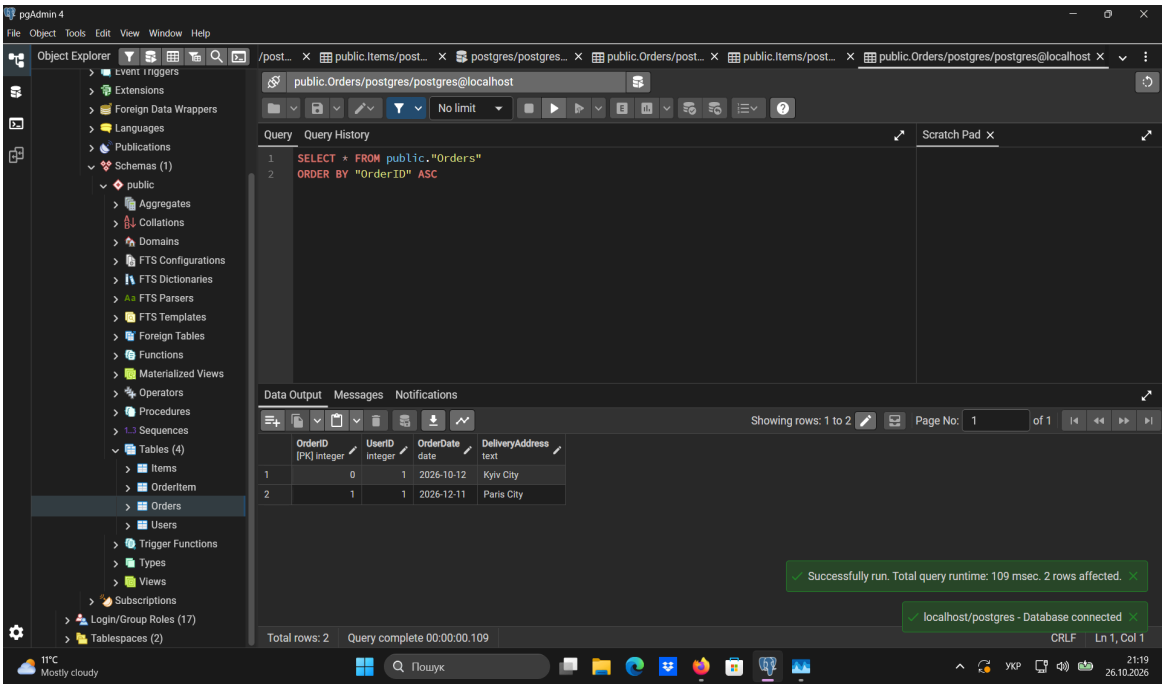
Таблиця Users.



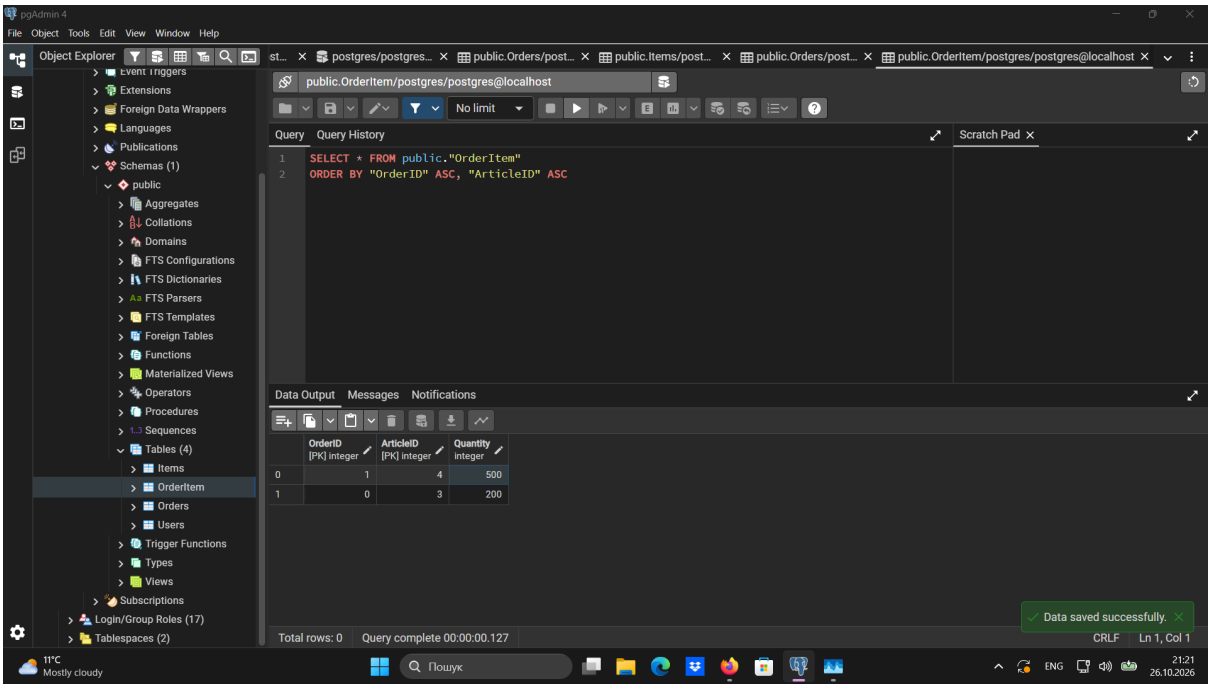
Таблиця Items.



Таблиця Orders.



Таблиця OrderItem.



Перевірка бізнес логіки створеної бази даних.